Do NLP Models Know Numbers? Probing Numeracy in Embeddings

Eric Wallace[*1], **Yizhong Wang**[*2], **Sujian Li**[2], **Sameer Singh**[3], **Matt Gardner**[1]
[1]Allen Institute for Artificial Intelligence
[2]Peking University
[3]University of California, Irvine
{ericw, mattg}@allenai.org, {yizhong, lisujian}@pku.edu.cn, sameer@uci.edu

Abstract

The ability to understand and work with numbers (numeracy) is critical for many complex reasoning tasks. Currently, most NLP models treat numbers in text in the same way as other tokens—they embed them as distributed vectors. Is this enough to capture numeracy? We begin by investigating the numerical reasoning capabilities of a state-of-the-art question answering model on the DROP dataset. We find this model excels on questions that require numerical reasoning, i.e., it already captures numeracy. To understand how this capability emerges, we probe token embedding methods (e.g., BERT, GloVe) on synthetic list maximum, number decoding, and addition tasks. A surprising degree of numeracy is naturally present in standard embeddings. For example, GloVe and word2vec accurately encode magnitude for numbers up to 1,000. Furthermore, character-level embeddings are even more precise—ELMo captures numeracy the best for all pre-trained methods—but BERT, which uses sub-word units, is less exact.

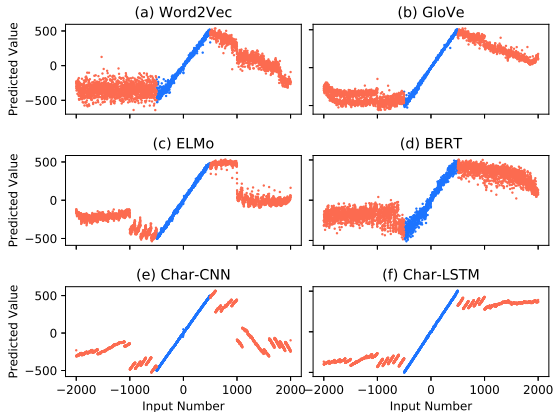

Figure 1: We train a probing model to decode a number from its word embedding over a random 80% of the integers from [-500, 500], e.g., "71" → 71.0. We plot the model's predictions for all numbers from [-2000, 2000]. The model accurately decodes numbers within the training range (in blue), i.e., pre-trained embeddings like GloVe and BERT capture numeracy. However, the probe fails to extrapolate to larger numbers (in red). The Char-CNN (e) and Char-LSTM (f) are trained jointly with the probing model.

1 Introduction

Neural NLP models have become the de-facto standard tool across language understanding tasks, even solving basic reading comprehension and textual entailment datasets (Yu et al., 2018; Devlin et al., 2019). Despite this, existing models are incapable of complex forms of reasoning, in particular, we focus on the ability to reason *numerically*. Recent datasets such as DROP (Dua et al., 2019), EQUATE (Ravichander et al., 2019), or Mathematics Questions (Saxton et al., 2019) test numerical reasoning; they contain examples which require comparing, sorting, and adding numbers in natural language (e.g., Figure 2).

The first step in performing numerical reasoning over natural language is *numeracy*: the ability to understand and work with numbers in either digit or word form (Spithourakis and Riedel, 2018). For example, one must understand that the string "23" represents a bigger value than "twenty-two". Once a number's value is (perhaps implicitly) represented, reasoning algorithms can then process the text, e.g., extracting the list of field goals and computing that list's maximum (first question in Figure 2). Learning to reason numerically over paragraphs with only question-answer supervision appears daunting for end-to-end models; our work seeks to understand *if* and *how* "out-of-the-box" neural NLP models already learn this.

We begin by analyzing the state-of-the-art NAQANet model (Dua et al., 2019) for DROP—testing it on a subset of questions that evaluate numerical reasoning (Section 2). To our surprise,

[*]Equal contribution; work done while interning at AI2.

Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, pages 5307–5315, Hong Kong, China, November 3–7, 2019. ©2019 Association for Computational Linguistics

the model exhibits excellent numerical reasoning abilities. Amidst reading and comprehending natural language, the model successfully computes list maximums/minimums, extracts superlative entities (argmax reasoning), and compares numerical quantities. For instance, despite NAQANet achieving only 49 F1 on the entire validation set, it scores 89 F1 on numerical comparison questions. We also stress test the model by perturbing the validation paragraphs and find one failure mode: the model struggles to extrapolate to numbers outside its training range.

We are especially intrigued by the model's ability to learn numeracy, i.e., how does the model know the value of a number given its embedding? The model uses standard embeddings (GloVe and a Char-CNN) and receives no direct supervision for number magnitude/ordering. To understand how numeracy emerges, we probe token embedding methods (e.g., BERT, GloVe) using synthetic list maximum, number decoding, and addition tasks (Section 3).

We find that all widely-used pre-trained embeddings, e.g., ELMo (Peters et al., 2018), BERT (Devlin et al., 2019), and GloVe (Pennington et al., 2014), capture numeracy: number magnitude is present in the embeddings, even for numbers in the thousands. Among all embeddings, character-level methods exhibit stronger numeracy than word- and sub-word-level methods (e.g., ELMo excels while BERT struggles), and character-level models learned directly on the synthetic tasks are the strongest overall. Finally, we investigate why NAQANet had trouble extrapolating—was it a failure in the model or the embeddings? We repeat our probing tasks and test for model extrapolation, finding that neural models struggle to predict numbers outside the training range.

2 Numeracy Case Study: DROP QA

This section examines the state-of-the-art model for DROP by investigating its accuracy on questions that require numerical reasoning.

2.1 DROP Dataset

DROP is a reading comprehension dataset that tests numerical reasoning operations such as counting, sorting, and addition (Dua et al., 2019). The dataset's input-output format is a superset of SQuAD (Rajpurkar et al., 2016): the answers are paragraph spans, as well as question spans, number answers (e.g., 35), and dates (e.g., 03/01/2014). The only supervision provided is the question-answer pairs, i.e., a model must learn to reason numerically while simultaneously learning to read and comprehend.

...JaMarcus Russell completed a 91-yard touchdown pass to rookie wide receiver **Chaz Schilens**. The Texans would respond with fullback Vonta Leach getting a 1-yard touchdown run, yet the Raiders would answer with kicker Sebastian Janikowski getting a 33-yard and a 21-yard field goal. Houston would tie the game in the second quarter with kicker Kris Brown getting a **53**-yard and a 24-yard field goal. Oakland would take the lead in the third quarter with wide receiver Johnnie Lee Higgins catching a **29-yard** touchdown pass from Russell, followed up by an 80-yard punt return for a touchdown.

Q: How many yards was the longest field goal? A: 53

Q: How long was the shortest touchdown pass? A: 29-yard

Q: Who caught the longest touchdown? A: Chaz Schilens

Figure 2: Three DROP questions that require numerical reasoning; the state-of-the-art NAQANet answers every question correct. Plausible answer candidates to the questions are underlined and the model's predictions are shown in bold.

2.2 NAQANet Model

Modeling approaches for DROP include both semantic parsing (Krishnamurthy et al., 2017) and reading comprehension (Yu et al., 2018) models. We focus on the latter, specifically on Numerically-augmented QANet (NAQANet), the current state-of-the-art model (Dua et al., 2019).[1] The model's core structure closely follows QANet (Yu et al., 2018) except that it contains four output branches, one for each of the four answer types (passage span, question span, count answer, or addition/subtraction of numbers.)

Words and numbers are represented as the concatenation of GloVe embeddings and the output of a character-level CNN. The model contains no auxiliary components for representing number magnitude or performing explicit comparisons. We refer readers to Yu et al. (2018) and Dua et al. (2019) for further details.

2.3 Comparative and Superlative Questions

We focus on questions that NAQANet requires numeracy to answer, namely *Comparative* and *Superlative* questions.[2] Comparative questions

[1] Result as of May 21st, 2019.

[2] DROP addition, subtraction, and count questions do not require numeracy for NAQANet, see Appendix A.

Question Type	Example	Reasoning Required
Comparative (Binary)	Which country is a bigger exporter, Brazil or Uruguay?	Binary Comparison
Comparative (Non-binary)	Which player had a touchdown longer than 20 yards?	Greater Than
Superlative (Number)	How many yards was the shortest field goal?	List Minimum
Superlative (Span)	Who kicked the longest field goal?	Argmax

Table 1: We focus on DROP *Comparative* and *Superlative* questions which test NAQANet's numeracy.

probe a model's understanding of quantities or events that are "larger", "smaller", or "longer" than others. Certain comparative questions ask about "either-or" relations (e.g., first row of Table 1), which test binary comparison. Other comparative questions require more diverse comparative reasoning, such as greater than relationships (e.g., second row of Table 1).

Superlative questions ask about the "shortest", "largest", or "biggest" quantity in a passage. When the answer type is a *number*, superlative questions require finding the maximum or minimum of a list (e.g., third row of Table 1). When the answer type is a *span*, superlative questions usually require an argmax operation, i.e., one must find the superlative action or quantity and then extract the associated entity (e.g., fourth row of Table 1). We filter the validation set to comparative and superlative questions by writing templates to match words in the question.

2.4 Emergent Numeracy in NAQANet

NAQANet's accuracy on comparative and superlative questions is significantly higher than its average accuracy on the validation set (Table 2).[3]

NAQANet achieves 89.0 F1 on binary (either-or) comparative questions, approximately 40 F1 points higher than the average validation question and within 7 F1 points of human test performance. The model achieves a lower, but respectable, accuracy on non-binary comparisons. These questions require multiple reasoning steps, e.g., the second question in Table 1 requires (1) extracting all the touchdown distances, (2) finding the distance that is greater than twenty, and (3) selecting the player associated with the touchdown of that distance.

We divide the superlative questions into questions that have number answers and questions with span answers according to the dataset's provided answer type. NAQANet achieves nearly 70 F1 on superlative questions with number answers, i.e., it can compute list maximum and minimums.

[3]We have a public NAQANet demo available https://demo.allennlp.org/reading-comprehension.

Question Type	Count	EM	F1
Human (Test Set)	9622	92.4	96.0
Full Validation	9536	46.2	49.2
Number Answers	5842	44.3	44.4
Comparative	704	73.6	76.4
Binary (either-or)	477	86.0	89.0
Non-binary	227	47.6	49.8
Superlative Questions	861	64.6	67.7
Number Answers	475	68.8	69.2
Span Answers	380	59.7	66.3

Table 2: NAQANet achieves higher accuracy on questions that require numerical reasoning (*Superlative* and *Comparative*) than on standard validation questions. Human performance is reported from Dua et al. (2019).

The model answers about two-thirds of superlative questions with span answers correctly (66.3 F1), i.e., it can perform argmax reasoning.

Figure 2 shows examples of superlative questions answered correctly by NAQANet. The first two questions require computing the maximum/minimum of a list: the model must recognize which digits correspond to field goals and touchdowns passes, and then extract the maximum/minimum of the correct list. The third question requires argmax reasoning: the model must first compute the longest touchdown pass and then find the corresponding receiver "Chaz Schilens".

2.5 Stress Testing NAQANet's Numeracy

Just how far does the numeracy of NAQANet go? Here, we stress test the model by automatically modifying DROP validation paragraphs.

We test two phenomena: larger numbers and word-form numbers. For larger numbers, we generate a random positive integer and multiply or add that value to the numbers in each paragraph. For word forms, we replace every digit in the paragraph with its word form (e.g., "75" → "seventy-five"). Since word-form numbers are usually small in magnitude when they occur in DROP, we perform word replacements for integers in the range [0, 100]. We guarantee the ground-truth answer is